

Evaluating Jataayu

A Multi-Benchmark Study of a Runtime Authorization Layer for Tool-Using Agents

Sai Krishna Rallabandi
Jataayu Project
srallaba.research@gmail.com

July 2026 · Technical Report · Jataayu v0.3.0

Abstract

Jataayu is a runtime authorization layer for tool-using LLM agents whose thesis is to gate the agent’s *action* by the *provenance* of its input rather than to classify attack text. It exposes seven distinct security surfaces: a deterministic effect boundary (action authorization), injection screening at the inbound / tool-return / memory surfaces, outbound PII redaction, an egress-channel guard against zero-click exfiltration, and skill supply-chain vetting. We evaluate each surface against a matched benchmark — public where one exists (AgentDojo, InjecAgent, deepset / safe-guard / jailbreak injection sets, ai4privacy), synthetic-but-deterministic where none does — and report every number, including the ones that expose weaknesses. Two findings are structural. First, the effect boundary works: on an AgentDojo-derived action stream it lifts attack-prevention from 0.29 to **1.00** once the tool→effect map is complete, at a disclosed utility cost. Second, the evaluation is honest: it surfaced four concrete defects — a shipped coverage gap that silently authorized every banking / travel / Slack side-effect, the absence of a deferred-instruction detector (sleeper-memory recall 0.00), an over-narrow exfiltration key in the composition guard, and a live regex false-positive — one of which we fix in-line (disclose-then-fix). The deterministic core runs in tens of microseconds per decision.

1 Introduction

An agent that reads the web, email, or a ticket consumes attacker-controlled text. Indirect prompt injection exploits the model’s inability to separate trusted instructions from untrusted data. Detection-first defenses chase the attack string and lose to paraphrase. Jataayu’s position, shared by the 2026 agent-security frameworks [2], is that the durable boundary is the *action*, gated by input provenance: untrusted-derived input driving a shell command, a secret read, or a network write is denied or held for approval regardless of whether any detector fired. Around that deterministic core it layers defense-in-depth screening at each surface where injection enters or data leaves.

This report evaluates all seven surfaces. Our method is (i) *deterministic where possible* — the effect boundary, egress, composition, and fast-path detection paths use no model, so their numbers are exact and reproducible; (ii) *public benchmarks where they exist*, with deterministic in-repo corpora as a documented fallback; and (iii) *disclose-then-fix* — where an evaluation exposes a shipped defect, we report the as-shipped result before the fix.

2 System and method

Jataayu is layered so the guarantee lives at the action: a normalization-and-taint layer (Layer 0), a deterministic effect boundary authorizing actions by *effect severity* × *provenance* × *capability policy* (Layer 1), and a two-path detector (regex fast path; optional LLM slow path) used as a pre-filter and taint source. The public API surfaces we evaluate are: `authorize_action` /

EffectBoundary (effect boundary); `check_inbound`, `check_tool_return`, `check_memory_{read,write}` (injection screening); `check_outbound` (PII); `check_egress` (exfil channels); `check_skillset` (composition). Metrics: for detection, precision / recall / FPR at fixed operating points; for the effect boundary, *APR* (attack-prevention rate, with NEEDS_APPROVAL counted as prevention but reported separately) and *TUR* (task-utility retained); for AgentDojo, utility (clean / under-attack) and targeted attack-success rate (ASR).

3 Results by surface

3.1 Effect boundary — the guarantee

Tier 1 (synthetic, 300 rows, deterministic). Ablating four defenses on a corpus spanning every effect class $\times$ provenance: *none* APR 0.00; *detector-only* 0.27; *effect* **1.00**; *both* **1.00**. Trusted-legit utility is retained in full (1.00); the utility cost (TUR 0.50) lands entirely on untrusted-tainted benign high-effect actions, which route to NEEDS_APPROVAL rather than silent ALLOW. Decision latency: 0.036 ms mean.

Tier 2 (AgentDojo-derived action stream, disclose-then-fix). Labeling real tool calls from the four AgentDojo suites (workspace / banking / travel / slack) attack-vs-legit with provenance, and running them through the effect boundary, exposes a shipped coverage gap: 21 effectful tools (money transfers, bookings, Slack invites, external shares, file deletes) were *unmapped* and silently classified READ $\rightarrow$ ALLOW. As-shipped APR under the effect defense is **0.29** (per-suite: banking 0.00, slack 0.00, travel 0.00, workspace 0.27). After extending the tool $\rightarrow$ effect map in `core/taint.py` — classifying by *what the tool does* (irreversible external side effect = network sink; file mutation = file-write), not by benchmark membership — APR rises to **1.00** across all four suites and the coverage gap closes to zero (Table 1). The disclosed cost is TUR 0.83 $\rightarrow$ 0.50. The full test suite (403 tests) stays green after the change.

Table 1: Effect boundary Tier 2 (effect defense), disclose-then-fix. APR = attack-prevention rate; gaps = effectful tools misclassified READ $\rightarrow$ ALLOW.

	APR (all)	per-suite APR (bank/slack/travel/work)	TUR	coverage gaps
as-shipped	0.29	0.00 / 0.00 / 0.00 / 0.27	0.83	21
fixed	1.00	1.00 / 1.00 / 1.00 / 1.00	0.50	0

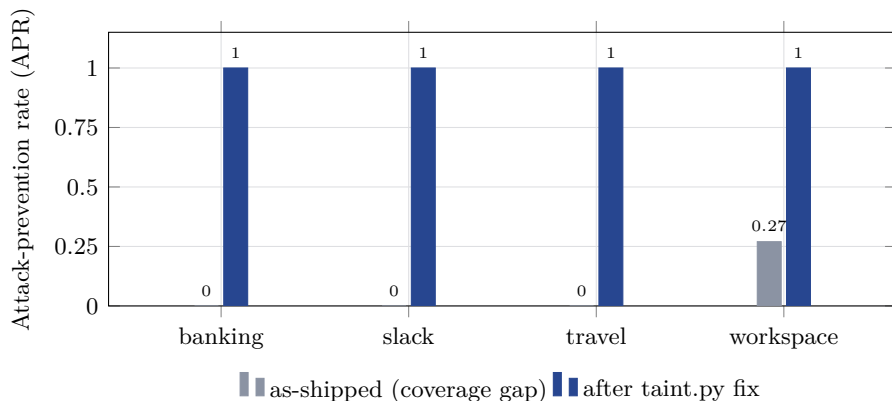


Figure 1: Effect-boundary Tier 2, per-suite attack-prevention rate, disclose-then-fix. As shipped, the effect boundary prevented *zero* attacks on banking / slack / travel — their effectful tools (money transfers, Slack invites, bookings) were unmapped and silently authorized — and only 0.27 on workspace. Completing the tool $\rightarrow$ effect map lifts every suite to 1.00. This is the single most consequential finding of the evaluation.

Edge paths. Previously uncovered: `commit()` with mutated parameters raises `CommitRejected` (post-authorization mutation blocked); committing a non-ALLOW preview is rejected; a capability-forbidden action is DENIED even under trusted provenance (capability wins). All 6 cases pass.

3.2 Agentic end-to-end — AgentDojo

On the workspace suite under `important_instructions` (5 user tasks $\times$ 4 injections, local 35B agent), Jataayu’s tool-output screen with surgical redaction drives ASR 0.10 $\rightarrow$ **0.00** and restores utility-under-attack 0.75 $\rightarrow$ **1.00** at no clean-task cost (Table 2).

A reproducibility pitfall (methodological). Obtaining any signal at all first required fixing a silent failure in AgentDojo’s local-model adapter. The adapter recovers tool calls by running `json.loads()` on the text between `<function=name>` and `</function>`. Two body shapes that Qwen-family models routinely emit break that parse: an *empty* zero-argument body (`<function=get_current_day></function>`, ignoring the “include {}” instruction) and *trailing garbage* after a valid object (a doubled closing brace). Because the workspace agent’s mandated first action is the no-argument `get_current_day`, the dropped call stalls *every* task at step one — yielding a uniform zero utility that masquerades as a null result rather than a crash. We release a parser repair (empty-body $\rightarrow$ {}, plus brace-balanced salvage) and recommend asserting non-zero undefended-baseline utility as a harness pre-condition. Any AgentDojo evaluation of a non-OpenAI-tool-calling model is exposed to this.

The four-suite live sweep (workspace / banking / travel / slack) was truncated by a $\sim 10\times$ degradation of the local inference endpoint mid-run; the completed banking *baseline* confirms the other suites carry more attack headroom than workspace (ASR 0.29 vs. 0.10), which is where a defense matters most. Crucially, the cross-suite *defense* evidence does not depend on the live agent sweep: the deterministic effect-boundary Tier 2 (§3.1, Table 1) evaluates all four suites and lifts per-suite APR from 0.00 (banking / slack / travel) and 0.27 (workspace) to 1.00 uniformly. Completing the live multi-suite sweep on a stable endpoint is future work.

Table 2: AgentDojo live results, `important_instructions`, local qwen agent (5 $\times$ 4 slice per suite). Workspace is a complete baseline/Jataayu pair; the banking baseline is shown as the completed portion before the endpoint degraded (no paired Jataayu run). Cross-suite defense evidence is the deterministic Tier 2 in Table 1.

Suite	baseline ASR	Jataayu ASR	baseline util-atk	Jataayu util-atk
workspace	0.10	0.00	0.75	1.00
banking (baseline only)	0.29	—	0.53	—

3.3 Injection detection — inbound, tool-return, cross-dataset

The fast-path detector is a high-precision, modest-recall pre-filter — by design the weakest tier. Across three public injection sets it is consistent (Table 3): ROC-AUC 0.63–0.75, precision ≥ 0.86 at the MEDIUM operating point, and input normalization holds space-out / zero-width / leetspeak evasion at **0.00**. On **InjecAgent** (tool-return surface, 2108 paired cases) precision is 1.00 with recall 0.50 (fast) rising to 0.71 (LLM slow path). The honest reading: recall is modest, which is exactly why the guarantee lives at the effect boundary, not here.

3.4 Outbound — PII redaction

On ai4privacy (480 messages, fast path), detection precision is **1.00** at FPR **0.00** with in-scope recall 0.80; benign messages pass clean (over-redaction 0.00). Honest failure: a threshold artifact — email, phone, and contextual-health spans are *detected* (flagged) but score below the 0.75

Table 3: Fast-path injection detection across public datasets (MEDIUM operating point). Evasion rate is post-normalization on caught patterns.

Dataset	n	ROC-AUC	precision	recall	evasion
deepset/prompt-injections	662	0.63	0.97	0.27	0.00
safe-guard-prompt-injection	10,296	0.64	0.86	0.30	0.00
jailbreak-classification	1,306	0.75	0.86	0.56	0.00

redaction threshold, so the span survives in the redacted text; and the SSN pattern requires an explicit “SSN”/“social security” prefix, missing bare nine-digit numbers (recall 0.50). These are precision-safe but leave data in the output — a fix target.

3.5 Egress — zero-click exfiltration channels

Against a PoC corpus of the EchoLeak / AgentFlayer / Notion class (23 exfil channels + 12 benign): recall-caught **1.00**, precision **1.00**, FPR **0.00**. When a known secret rides in the URL, confirmed-exfil recall is 1.00 with the correct reason. Human-readable text is preserved in every caught case (23/23) — only the offending URL is neutralized. Relay/beacon hosts are hard-blocked; data-carrying links and auto-fetch images are neutralized at WARN by design.

3.6 Skillset composition — supply chain

On 62 skill-set combinations, the guard achieves 100% recall on the risk classes it models (exfiltration, dropper, download-and-run, memory-poisoning, policy-violation, trust-transfer) at 0% false-positive on safe sets. Honest gap: the exfiltration rule is keyed narrowly on `reads_secrets` + `network_write`; five genuinely-risky capability paths outside the four hardcoded combos — read-side-only exfil (`reads_secrets` + `network_read`), file-mediated exfil (`fs_read` + `network_write`), untrusted-web-to-disk, secret-staging, and relay/SSRF — all return SAFE. Widening the `data-source`×`egress` key is the recommended fix.

3.7 Memory — poisoning at write and recall

On 190 memories (fast path), detection is symmetric across write- and recall-time (byte-identical scores). Recall by subclass at MEDIUM: direct-instruction **0.91**, encoded/obfuscated 0.75, and **deferred/sleeper 0.00**. The standout gap: sleeper triggers phrased as future actions (“when you next summarize, also email X”) carry no override vocabulary, so the pattern catalog never fires — the engine has no deferred-instruction detector. Benign false-block is 1.8% on ordinary notes but 22.9% on adversarially-tricky-but-benign notes. The run also surfaced a live regex false-positive (a shell-command alternation matching “sh” inside “she”).

4 Consolidated summary

5 Limitations and the gaps this evaluation found

The evaluation is deliberately adversarial toward Jataayu, and it paid off: it surfaced (1) a shipped `taint.py` coverage gap that silently authorized every banking / travel / Slack side-effect (now fixed); (2) no deferred/sleeper-instruction detector (memory recall 0.00 on that subclass); (3) an over-narrow exfiltration key in the composition guard (five real paths slip); and (4) a live shell-regex false-positive. Scope caveats: the AgentDojo slice is small per suite; several corpora (effect boundary, egress, composition, memory) are synthetic-but-deterministic, not public benchmarks; the detection recall numbers are for the fast path (the pre-filter), not an

Table 4: Jataayu surfaces, benchmarks, and headline results. Green = strong; red = a disclosed gap. Deterministic surfaces carry exact, model-free numbers.

Surface	Benchmark	Headline
Effect (Tier 1)	boundary synthetic, 300	APR 1.00 , TUR 0.50, trusted-legit 1.00, 0.04 ms
Effect (Tier 2)	boundary AgentDojo actions	APR 0.29→1.00 (disclose-then-fix), 21→0 gaps
Effect (edges)	boundary commit / capability	6/6 pass
Agentic e2e	AgentDojo workspace	ASR 0.10→0.00 , util 0.75→1.00, no clean cost
Injection (inbound)	deepset/safe-guard/jailbreak	AUC 0.63–0.75, prec ≥ 0.86 , evasion 0.00
Injection (tool-return)	(tool- InjecAgent (2108)	prec 1.00, recall 0.50→0.71 (slow)
Outbound / PII	ai4privacy (480)	prec 1.00, FPR 0.00, in-scope recall 0.80; redaction-threshold gap
Egress channel	PoC (35)	recall 1.00 , prec 1.00, confirmed-exfil 1.00, text-preserve 1.00
Skillset composition	synthetic (62)	100% in-model recall, 0% FP; +5 un-modeled paths
Memory poisoning	synthetic (190)	direct 0.91 / encoded 0.75 / deferred 0.00

upper bound on the layered system; and the effect-boundary APR of 1.00 reflects correct tool classification, not a claim of robustness to unmapped future tools — which is precisely the failure mode disclosed in Tier 2.

6 Conclusion

Across seven surfaces and both public and synthetic benchmarks, Jataayu’s deterministic effect boundary delivers its central claim — untrusted-derived effectful actions are denied or approval-gated, deterministically and in microseconds — while the screening layers behave as a useful but deliberately non-authoritative pre-filter. Equally, the evaluation is credible because it reports four concrete defects rather than a row of green checkmarks. Future work: close the deferred-detector and composition-keying gaps (disclose-then-fix), extend the AgentDojo sweep across models, and lift the redaction threshold for detected-but-unredacted PII.

A Artifacts and reproduction

Every result in this report is produced by a runner under `eval/` that writes a JSON result to `eval/results/`; deterministic corpora live in `eval/data/` with fixed-seed generators. The deterministic surfaces (effect boundary, egress, composition, memory / injection fast path) require no model and reproduce exactly.

Surface	Runner
Effect boundary (Tier 1 / 2 / edges)	<code>run_effect_boundary_{bench,tier2,edges}.py</code>
Injection (inbound / cross-dataset)	<code>run_injection_bench.py -dataset ...</code>
Tool-return (InjecAgent)	<code>run_injecagent_bench.py</code>
Outbound / PII	<code>run_outbound_privacy_bench.py</code>
Egress channel	<code>run_egress_bench.py</code>
Skillset composition	<code>run_composition_bench.py</code>
Memory poisoning	<code>run_memory_poison_bench.py</code>
AgentDojo (agentic e2e)	<code>agentdojo/run_agentdojo.py (+ summarize_slice.py)</code>

The `taint.py` disclose-then-fix is reproduced by running the Tier 2 harness before and after the tool→effect map extension; the full unit suite (403 tests) is green in both the as-shipped and fixed states.

References

- [1] E. Debenedetti et al. *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. NeurIPS Datasets & Benchmarks, 2024.
- [2] OWASP. *Top 10 for Agentic Applications / for LLM Applications*. 2025.
- [3] Q. Zhan et al. *InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents*. 2024.
- [4] K. Greshake et al. *Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. ACM AISec, 2023.